



FIFTH ANNIVERSARY

FIVE OF DIGIT'S TECH DESIRES ■ FIVE OF DIGIT'S TECH DESIRES ■ FIVE OF DIGIT'S TECH DESIRES ■ FIVE OF DIGIT'S TECH DESIRES

UtopiOS

I never want to bother with drivers or compatibility again. Ever!

Nimish Chandiramani

It's a beautiful day outside, but you don't care. It's a beautiful day inside, too—the installer for your much coveted Linux distro approaches the hallowed 100 per cent mark as you rub your hands in glee. You reboot into your new OS, and oh horror! Your Ethernet card isn't supported! How are you supposed to connect to the Internet? As you poke around trying to figure out what to do, you find that even more of your hardware turns up as "Unknown Device." Countless profanities and numerous clumps of ripped-out hair later, you're back to good old Windows, and have vowed never to go a-wandering for another OS again.

Meeting Ground

We are, of course, no strangers to the desire for our applications to run on all OSes, irrespective of which one it was written for—Windows, Linux, Mac, even Amiga. And to a very reasonable extent, this is quite possible, thanks to Compatibility Layers such as Wine, which let you run Windows applications on Linux. Applications give Wine the same instructions that they do a Windows PC, and Wine translates those instructions to make them understandable to the Linux PC. Ditto Darwine, the Wine that will let Mac users run Windows applications.

While everything seems all hunky-dory on the application front, there's a deeper, more painful issue to be tackled—drivers. Emulation takes a big bite out of system performance, and for something as critical as device drivers, the very idea of using emulation is horrifying. The only answer is to write drivers specifically for the OS. Right.

The Hidden Truths

A device driver is a piece of software that lets the operating system's kernel talk to your PC's hardware. The driver gives the OS a Hardware Abstraction Layer (HAL), which basically obviates the need for the OS to know exactly what hardware it's running on. All it needs to do is tell the driver what it wants the device to do—the rest is up to the driver.

Things start to smell a little fishy here—if drivers are separate from the main OS, why are they tied so closely to it? A printer driver, for example, will have a function called *print*, which the OS can use every time it wants to print something. Since all device drivers are written in C, they should work with anything that calls their functions. Believe it or not, we already live in a world where theoretically, device drivers need to be written only once—they can then be used in Windows, Linux, Unix, and any other OS. This means that you wouldn't need to go hunting for Linux drivers for your hardware—if it was supported in one OS, it's supported in all!

The Bitter Pill

The reason we struggle so much for drivers is that there's no *standard* way for operating systems to talk to drivers—Windows may say *print()*, but Mac may say *printme()* for the same function. Windows has the WDM (Windows Driver Model) standard, but it's just designed to work for all versions of Windows. Windows drivers also come bundled with a lot of (probably unnecessary) additions. No real printer driver is 25

MB—it's the additional utilities like ink-level monitoring et al that add to its bulk.

Linux, on the other hand, has no real driver standard—drivers can even stop functioning after a kernel update! Moreover, each distributor might modify the kernel a little—imagine having a Linux driver that doesn't even work with all distros. Eeyugh!

Are We There Yet?

Not too long ago, we stumbled upon a project called DriverLoader (www.linuxiant.com) that lets you use Windows network drivers as is in Linux, and without emulation, too. It builds a wrapper around the driver that simply links the Linux instruction to the instruction in the driver as written for Windows. If this were to keep developing, we'd soon have a Linux that didn't care about drivers—"Oh, I see you have Windows drivers for this device. Oh well, that will do too."

The possibility of this desire fulfilled—to never have to be bound to an OS—is eerily possible; so much so, in fact, that it's a wonder nobody thought of it yet. Or have they?

ReactOS (www.reactos.org) is a project that is working on making an open source Windows clone. It can run Windows applications and use Windows device drivers with few or no hang-ups. Later versions, the team promises, will also let you use applications and drivers written for the Linux and Unix platforms, making it possible to even run Macintosh applications, though probably not out-of-the-box just yet. Could this be the next big thing—the OS that runs *everything*? We itch in anticipation.

Of course, there's a better way out than developing a whole new OS—a single, universal standard for developing device drivers that all OSes will implement and all device manufacturers will use. That's on our wish-list too—right up there with world peace. ■

nimish_chandiramani@thinkdigit.com

Illustration Pravin Warhokar

